claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-ae6fe818350b7f98a.jsonl`
- SHA-256: `a9e20b41e0e4b9d6ee5eab081f0049a81ac0a838feef0bc8c5ce7410184f0faf`
- Source modified: `2026-07-08T06:38:13+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:36:06.764Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.
A prior reviewer raised this finding on the DESIGN-DOC ACCURACY (docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md) dimension:
  SEVERITY: low
  FILE: docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:13
  SUMMARY: §0's '~175 s of idle GPU per run (~$0.08?0.16)' upper bound is not derivable from the §4.2 cost model.
  CLAIMED FAILURE: 175 s at the doc's own L4 Job rate ($0.000474/s) = $0.083; at the warm-GPU-Service rate ($0.000349/s) = $0.061; at GPU-only ($0.00022404/s) = $0.039. The full upload-onto-L4 topology delta in §4.2 is only +$0.13. No stated rate × 175 s reaches $0.16, so the upper bound is unsupported by the model.
  PROPOSED FIX: Narrow to ~$0.08?0.13, or show the derivation (e.g. state the rate and duration that produce $0.16).

Adversarially VERIFY it against the real code/doc in C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. assistant (2026-07-08T06:36:10.039Z)

I'll verify this finding against the actual doc. Let me read the relevant sections.

3. user (2026-07-08T06:36:20.061Z)

L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the contr

> **Status:** `DRAFT` ? **owner-review** (recommends *against* the epic's proposed direction #1; see §0/§8) · **Owner:** Avi · **Created:** 2026-07-08 · **Last updated:** 2026-07-08
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524) (epic) · **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521) (`phase_placement`) · **Motivated by:** [#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that surfaced the data-movement finding)
> **Relation to existing docs:** extends [COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not** supersede them ? it answers the *topology* question they left open and sequences the migration on top of `phase_placement`.
> **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog), `[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]` (proposed). Not legal/financial advice.

---

0. TL;DR

The 2026-07-08 live CUJ moved one 1.83 GiB clip **3×** (upload?bucket, bucket?control-plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for **~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by **uploading directly onto the L4** and async-copying to the bucket. **This design recommends *against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a **minutes-long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a durability exposure window instead of closing one.

The cheaper, more durable, already-ratified path is the opposite: **keep the upload streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ? which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual "~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane **never downloading the file at all** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero idle-GPU cost** and **zero exposure window**.

Actionable now (this PR): the design doc + **win #2** (drop the redundant control-plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.

1. Problem & goal

Why now. Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 `[verified]`) produced two findings:

1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? `storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect (`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).

2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

The concrete outcome we want. The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls the full file**. Every move must be **reversible by config** with a **bucket-mediated fallback**, and land as small PRs on top of `origin/master`.

The hard question this doc must resolve (compute topology). "Upload directly to the L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch, no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a hybrid** presign/redirect. §4 answers it with numbers.

Read to get current: [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the `phase_placement` knob this sequences on, [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

2. Decisions locked

#	Decision	Source / date	Implication
L1	**Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed for durability.	this doc `[design]` + #480 `[verified]`	No exposure window; upload cost stays on the cheap I/O path.
L2	**Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via `phase_placement` (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)** (Jobs, not Services).	#521 `[verified]` + D7/D16	The "L4-primary" vision is realized by **config flips**, not a rearchitecture.
L3	**Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash).	this doc `[design]`	The CP stops pulling the full file when validation is delegated; movement ? 3×?2×.
L4	**Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.** Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300 runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at sustained load.	§4.2 cost model `[design]`	Preserves per-run billing (D7/D8); avoids ~$382?637/mo idle.
L5	**Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN pull at the cost of ~175 s idle GPU/run **and** opens a durability window.	§4.3 `[design]`	The epic's proposed direction #1 is not adopted; owner sign-off requested (§8).

4. assistant (2026-07-08T06:36:22.995Z)

Now let me read §4.2 and §4.3 with the cost model.

5. user (2026-07-08T06:36:32.026Z)

59:## 4. Design / architecture ? resolving the topology question
60:
61:### 4.1 The measured pipeline (evidence base) `[verified]`
62:
63:From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s / 4K-class):
64:
65:| Phase | Wall | Machine | GPU needed? | Notes |
66:|---|---|---|---|---|
67:| Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ? **owner bandwidth**, not infra |

68:| Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to scratch (`acquire_local_input` + re-hash) |
69:| Validation (#512 one-pass) | **183 s** | control-plane 2 vCPU | no (CPU-bound) | scene_cut/blur/relevance shared decode |
70:| Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse + torch/cuda init (cold start) |
71:| Detection (WASB) | **161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound phase |
72:| Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73:| Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) | 11× 4K re-encode+watermark |
74:
75:**Two facts that drive the whole design:**
76:- **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about **co-locating CPU work next to the GPU on a box that's already up and already holding the bytes**, not about the GPU per se.
77:- **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
78:
79:### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08) `[verified]` rates / `[design]` per-run
80:
81:Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-lang-client-0306026826`):
82:
83:| Resource | Rate | Source |
84:|---|---|---|
85:| Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog `[verified]` |
86:| Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87:| Cloud Run memory | $0.0000024 / GiB·s | Catalog `[verified]` |
88:| ? **L4 Job @ 8 vCPU / 32 GiB (running)** | **? $1.705 / hr** ($0.000474/s) | derived `[design]` |
89:| GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **~$637 / mo** | Catalog `[verified]` |
90:| GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog `[verified]` |
91:| Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr = **~$917 / mo** idle | derived `[design]` |
92:| Balanced PD | $0.11 / GiB·mo | Catalog `[verified]` |
93:
94:**Per-run GPU-allocated cost (the term that varies by topology):**
95:
96:| Topology | GPU-allocated time / run | GPU cost / run | ? vs today | Concurrency wall? |
97:|---|---|---|---|---|
98:| **Today** (detect-only on L4; validate+stitch on CP) | ~222 s (cold+detect+io) | **~$0.105** | ? | **Yes** ? ~16 min CP CPU/run |
99:| **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60 + detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100:| **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) | **~$0.231** | **+$0.13** | No, but idle-GPU tax |
101:| **Persistent GCE VM** (on-demand) | billed 24/7 regardless | **$637/mo flat** | breaks per-run billing | No |
102:
103:**Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51 %** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7** `[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our volume ? the ~$382?637/mo would be almost pure waste.
104:
105:### 4.3 The three topology options, scored
106:
107:**Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.**
108:- *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1** removes cold start but pins a **~$917/mo warm idle GPU**.

109:- *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175 s upload** ? the idle-GPU tax of §4.2 on *every* request, plus the warm-idle bill if min-instances>0.
110:- *Ops/parity:* **contradicts D16(a)** (Jobs, not Services) and **D7** (scale-to-zero) ? abandons the `JobWaiting`/`claim_due_job`/bucket-poll machinery that's shipped and tested. A Service also needs request-timeout/concurrency tuning the batch Job sidesteps.
111:- **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.
112:
113:**Option B ? GCE L4 VM (persistent, reliable PD).**
114:- *Reliability:* real, attached, reliable PD ? the owner's stated appeal. But **the bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.
115:- *Cost:* **$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.
116:- *Ops:* we now **own a VM** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.
117:- **Verdict: reject at current volume; owner-gated future ADR** if sustained load crosses the §4.2 breakeven.
118:
119:**Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.**
120:- *The fatal timing problem:* compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up *to receive* the upload means the GPU is **allocated and idle for the entire 175 s upload** ? the §4.2 idle tax, same as A. Upload and compute **cannot overlap**.
121:- *Durability:* introduces an **exposure window** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **loses the source** unless the client re-uploads (see §4.5).
122:- **Verdict: reject.** It pays the idle-GPU tax *and* takes on a durability window, to save a transfer that is ~free and seconds-fast.
123:
124:**Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).**
125:- Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase_placement` (#521). Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.
126:- *Data movement:* **1 client upload + 1 intra-region LAN pull** ? the floor. (Detection and validation/stitch all run in **one** Job execution, so it's one pull, not two.)
127:- *Cost:* +$0.04/run GPU vs today, **?~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.
128:- *Durability:* **no exposure window** ? the bucket copy exists before the Job starts.
129:- *Ops/parity:* **honors D4/D7/D16**; reuses the shipped dispatch/poll path; every phase is a **config flip with an in-process fallback** (#521's `_dispatch_*_remote` already logs-and-falls-back to in-process when the box isn't cloud-wired).
130:
131:> **The one-line intuition:** the bucket *is* the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.
132:
133:### 4.4 The minimal control-plane surface (the "thin router")
134:
135:After A?, the control-plane does **only** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):
136:
137:| Keep on the control-plane | Why it must stay | Code anchor `[verified]` |
138:|---|---|---|
139:| **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |
140:| **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the durable-copy producer | `backend/api/uploads.py` |
141:| **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the async job lifecycle (`JobWaiting`/`claim_due_job`) | `backend/orchestration.py`, `backend/compute/cloud_run.py` |
142:| **DB writes** (jobs, videos, `play_segments`, validation verdict) | The control-plane owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database*.py` |

143:| **SPA serving + job status** (`/api/jobs/{id}` polling) | The UX surface | `backend/api/jobs.py`, `backend/ui/` |
144:| **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP just signs | `backend/main.py::get_highlights` |
145:| **~~Validation~~** | ? **L4** via `phase_placement.validation` (#521) | delegated |
146:| **~~Stitch/compile~~** | ? **L4** via `phase_placement.compile` (#521) | delegated |
147:| **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |
148:
149:### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future upload-onto-L4 path is ever adopted)
150:
151:The recommended A? design **has no async-copy step** (the bucket copy is the *upload*, produced before compute). This section specs the model the epic asked for, so that if the owner overrides §8 and adopts upload-onto-L4, the durability semantics are pinned rather than improvised:
152:
153:- **When the mirror runs:** kick a background `storage.put(local ? gs://?/uploads/?)` **immediately on upload-complete** (not "during segmentation" ? starting it later only lengthens the exposure window; there's no compute reason to delay it). Run it concurrently with validation/detect.
154:- **Failure/resume:** retry with capped exponential backoff; the mirror is **idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the source is still on the client). **Never** report success until the mirror is **confirmed** (object exists + size/md5 match).
155:- **Exposure window:** upload-complete ? mirror-confirmed, the **only** durable copy is L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds `[design]`). A preemption in-window = re-upload. **This window is the entire reason A? is preferred** ? A?'s window is *zero*.
156:- **Ack contract:** the client keeps the source until the API returns `mirror_confirmed:true`; only then may it release it. (A? needs no such contract ? bucket-first.)
157:
158:## 5. Phased migration ? every step a config flip with a bucket-mediated fallback

6. assistant (2026-07-08T06:37:32.668Z)

Let me confirm the exact line of the claim and check for any other rate in the doc that could justify $0.16.

7. user (2026-07-08T06:37:36.628Z)

13:The 2026-07-08 live CUJ moved one 1.83 GiB clip **3×** (upload?bucket, bucket?control-plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for **~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by **uploading directly onto the L4** and async-copying to the bucket. **This design recommends *against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a **minutes-long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a durability exposure window instead of closing one.
15:The cheaper, more durable, already-ratified path is the opposite: **keep the upload streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ? which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual "~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane **never downloading the file at all** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero idle-GPU cost** and **zero exposure window**.
39:| L4 | **Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.** Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300 runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at sustained load. | §4.2 cost model `[design]` | Preserves per-run billing (D7/D8); avoids ~$382?637/mo idle. |

40:| L5 | **Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | §4.3 `[design]` | The epic's proposed direction #1 is not adopted; owner sign-off requested (§8). |
52:| **D7** | **Scale-to-zero, no warm pool** (a ~$500/mo idle GPU "would break per-video billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-ADR territory (§4.2). |
77:- **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
86:| Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
91:| Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr = **~$917 / mo** idle | derived `[design]` |
99:| **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60 + detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100:| **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) | **~$0.231** | **+$0.13** | No, but idle-GPU tax |
103:**Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51 %** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7** `[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our volume ? the ~$382?637/mo would be almost pure waste.
108:- *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1** removes cold start but pins a **~$917/mo warm idle GPU**.
109:- *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175 s upload** ? the idle-GPU tax of §4.2 on *every* request, plus the warm-idle bill if min-instances>0.
111:- **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.
115:- *Cost:* **$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.
120:- *The fatal timing problem:* compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up *to receive* the upload means the GPU is **allocated and idle for the entire 175 s upload** ? the §4.2 idle tax, same as A. Upload and compute **cannot overlap**.
122:- **Verdict: reject.** It pays the idle-GPU tax *and* takes on a durability window, to save a transfer that is ~free and seconds-fast.
127:- *Cost:* +$0.04/run GPU vs today, **?~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.
131:> **The one-line intuition:** the bucket *is* the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.
176:- **A green suite does not prove the cloud path** ? the tests are fully-mocked/offline (a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are validated **in the cost model + code paths**, not on live infra. A live CUJ on rev ? `00020` (owner-side, D17 budget) is required to *confirm* the gap is gone and the movement is 2×.
199:1. **Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it; this doc recommends against it with the §4.2/§4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. *If you still want upload-onto-L4* (e.g. for a future no-bucket edge deployment), §4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).

8. assistant (2026-07-08T06:38:05.333Z)

I have verified the arithmetic against the actual cost model. Let me state the conclusion.

The claim is at `docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:13` (§0): "~175 s of idle GPU per run (~$0.08?0.16)".

Every rate in the §4.2 model (lines 83?92), applied to 175 s:
- L4 Job @ 8 vCPU/32 GiB, $0.000474/s (the highest GPU-allocated rate in the doc): 175 × 0.000474 = **$0.083**
- Warm GPU Service, $1.256/hr = $0.000349/s: 175 × 0.000349 = $0.061
- L4 GPU-only, $0.00022404/s: 175 × 0.00022404 = $0.039
- GCE g2 on-demand, $0.872/hr = $0.000242/s: 175 × 0.000242 = $0.042

The model's own idle-tax figure confirms this: §4.2 line 100 shows Upload-onto-L4 = $0.231 vs #521's $0.148 (line 99), a delta of **$0.083** ? i.e. exactly 175 s × the Job rate. Even the most generous multi-baseline reading (delta vs today, line 100 = **+$0.13**) does not reach $0.16. To hit $0.16 at 175 s you'd need $0.000914/s, a rate that appears nowhere in the doc. The $0.16 upper bound looks like a conflation with the #521 total per-run figure ($0.148 ? $0.15), which is a different quantity, not the idle tax.

So the §0 upper bound is genuinely non-derivable from the §4.2 model it claims to summarize ? a real internal inconsistency in a doc whose honesty note promises `[design]`/`[verified]` reconciliation. The lower bound ($0.08) is supported; only the upper bound is wrong.

9. user (2026-07-08T06:38:13.432Z)

Structured output provided successfully